

Exploring ScrumBut—An empirical study of Scrum anti-patterns



Veli-Pekka Eloranta^{a,*}, Kai Koskimies^b, Tommi Mikkonen^b

^a Vincit Ltd, Tampere, Finland

^b Department of Pervasive Computing, Tampere University of Technology, Tampere, Finland

ARTICLE INFO

Article history:

Received 23 March 2015

Revised 26 October 2015

Accepted 1 December 2015

Available online 14 December 2015

Keywords:

Software development

Scrum

Anti-patterns

Empirical study

ABSTRACT

The wide-spread adoption of the agile movement has rapidly changed the landscape of software industry. In particular, Scrum is an agile process framework that has become extremely popular in industry. However, the practical implementation of Scrum in companies rarely follows the text book ideals, as companies often deviate from the proposed Scrum practices for various reasons. While some deviations may be well motivated and reasonable, companies can also be tempted to adjust Scrum for the company without clearly understanding the consequences of the deviations. In this paper our aim is to identify ways of potentially harmful mishandling of Scrum in industry based on empirical data collected in semi-structured interviews involving 18 teams in 11 companies. The (mal)practices that were identified at least in three different teams are presented in a semi-formal manner as anti-patterns. The study resulted in 14 anti-patterns that express the context of the deviation, the deviation itself, the broken core principles of Scrum, and the possible consequences of the deviation. In addition, where available, we have included company recommendations regarding the deviations. Furthermore, we identify potential risk areas in Scrum based on an analysis of the relationships between anti-patterns and Scrum concepts.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

The wide-spread adoption of the agile movement—culminating in many ways in the Agile Manifesto published in 2001 [1] at the level of ideals—has taken off in the software industry in a rapid pace. However, when inspected more closely and in more detail, by phrase “we are agile in software development”, companies often mean simply, “we are using some practices or ideas of Scrum”. This is commonly referred as ScrumBut [14,18].

Scrum is a simple, iterative framework for project management [12,13,15]. At present, the adoption of Scrum has progressed in several organizations so far that it is possible to start collecting empirical data on the use of Scrum. To consolidate accurate, practical knowledge that is relevant to companies developing software, it is important to gather data on the Scrum practices in action for determining which parts of the methodology pose problems in industry. In particular, it is useful to understand how and why companies actually deviate from recommended Scrum way of working.

In this paper, our goal is to get empirical data on ScrumBut, that is, on the deviations of the recommended Scrum practices in companies that rely on Scrum in their software development, and present

those deviations systematically as anti-patterns [2]. In general, anti-patterns are common practices that are seen initially convenient and appropriate, but that are typically harmful in the long run and should therefore be avoided. An anti-pattern is often associated also with an alternative, recommended practice, which is more appropriate in most of the cases. Anti-patterns are presented using a well-defined format, identifying different aspects of the potentially harmful practices and making them easier to recognize, understand, and avoid.

The anti-pattern concept fits nicely our aim to describe in a semi-formal manner the constituents of ScrumBut. Since Scrum is a comprehensive framework rather than just a collection of practices, deviations from Scrum recommendations are, in general, regarded potentially harmful. Such a deviation may endanger the realization of the goals of Scrum as a whole. However, it should be emphasized that we do not present conclusions about the harmfulness of the found deviations based on empirical data. Indeed, sometimes a company has resorted to a deviation for good reasons, and this deviation is well justified. We have taken this into account by considering also these cases in each anti-pattern. In general, in this context the alternative “good” practice which is a part of an anti-pattern is the recommended text book Scrum practice.

The focus of this work is in the identification and presentation of the Scrum deviations based on empirical data. Essentially, our aim is to shed light on the relatively fuzzy concept of ScrumBut by identifying and exploring actual Scrum deviations in their context. We see the usefulness of this work mainly in two directions. First, a

* Corresponding author. Tel.: +358445570571.

E-mail addresses: veli-pekka.eloranta@iki.fi (V.-P. Eloranta), tommi.mikkonen@tut.fi (T. Mikkonen).

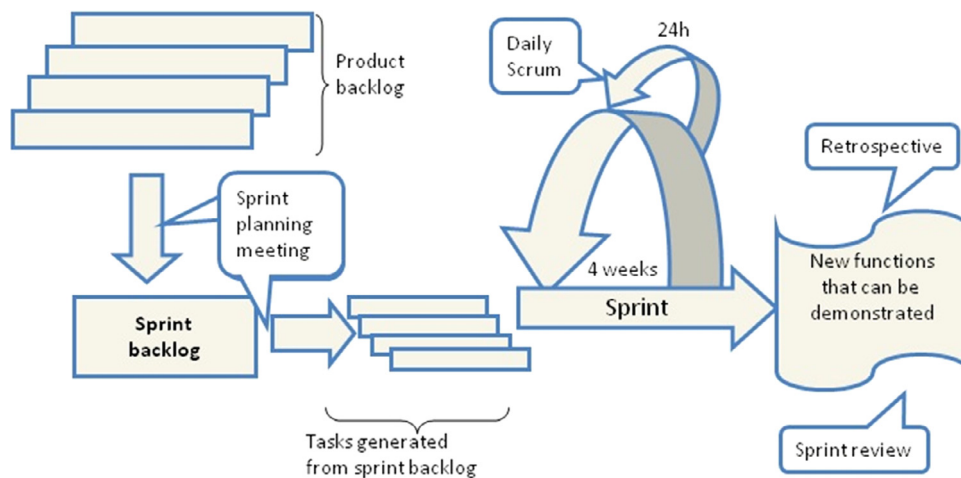


Fig. 1. Scrum in a nutshell.

company starting to use Scrum should be aware of common deviations that may look reasonable but that can be actually harmful. At least, the company should understand the context and the issues involved typically in such deviations. Second, understanding well-motivated deviations from the standard, text-book Scrum provides information for refining the methodology to fit the purposes of companies developing software.

To mine typical Scrum deviations in industry, we have conducted a survey of Scrum related practices in IT companies that use Scrum. The results of the survey are presented in this paper in terms of basic Scrum practices. For each basic practice, we summarize the contents of the interviews, especially concentrating on the ways the practices have been realized in the companies and possible deviations from Scrum recommendations. Then, we formulate the identified deviations from Scrum as anti-patterns using a format specifically designed for this purpose. These anti-patterns, together with the empirical data supporting them, are the main contribution of this paper.

The rest of this paper is structured as follows. Section 2 introduces Scrum, which forms the background of this paper. Section 3 provides the basic information concerning the survey method. In Section 4 we summarize the results of the survey structured according to the main Scrum concepts, discussing the extent to which companies follow the related Scrum practice and the possible deviations. Section 5 condenses the contribution of this paper by formulating the identified deviations as anti-patterns. Section 6 discusses the limitations of this study, and Section 7 summarizes previous work on the usage of Scrum in industry, with comparisons to our work. Towards the end of the paper, Section 8 introduces some directions for future work, and Section 9 draws final conclusions.

The paper is based on an earlier conference paper [17]. The extensions included in this paper include a brief introduction to Scrum to make the paper understandable for a wider audience, four new anti-patterns not published in the original paper, identified after further analysis of the same data, and an improved anti-pattern format with a company recommendation field to record possible solutions to problems as identified by the companies. In addition, the relationships between anti-patterns and Scrum concepts have been analyzed and visualized with a graph. Finally, the text has in general been revised to make it better suited for journal readers.

2. Background

Within a relatively short amount of time, an increasing number of organizations have adopted an agile way of working. When considering actions taking place inside individual companies and development teams, agile development usually means that they are using at

least some of the practices included in Scrum [15], a commonly used agile development framework (Fig. 1).

When using Scrum, incoming requirements are first stored in the Product Backlog in the form of so-called Product Backlog Items. These items are to be implemented in terms of Sprints, or iterations whose usual length is few weeks. For each Sprint, a collection of Product Backlog Items are selected for that particular Sprint and refined into a task list called the Sprint Backlog, consisting of Sprint Backlog Items. Collectively, these tasks form an implementation plan for the selected Product Backlog Items, provided in sufficiently concrete terms that it is possible to produce accurate work estimates for all of them.

After the refinement process, Sprint Backlog Items are implemented by executing the Sprint. The development team that executes the Sprint can select the best possible tools for the job, and consequently several techniques are commonly applied. Tools that are usually needed include, for example, support for version control, managing Backlog Items, testing facilities, and bug tracking systems. The details of the tool set fall beyond the scope of this paper.

Upon the completion of each Sprint, a complete system that can in principle be delivered to clients is available. In order to learn from each Sprint, a Retrospective is carried out to consider how the ways of working could be improved.

Only three roles are defined in Scrum: the Product Owner is responsible for managing the Product Backlog, the Team consists of developers responsible for executing the Sprint, and the duty of the Scrum Master is to eliminate any emerging impediment and enforce the Scrum process.

Although the above ideals form a coherent framework for executing development tasks, it is common that organizations follow them only partially or in a fashion where some of the practices are totally omitted. Omitting some Scrum practices or changing them in a way that is not aligned with the Scrum guidelines is commonly referred to as ScrumBut [18]. Such misalignment has been blamed to be a source of missing many of the benefits of Scrum [19], although little scientific evidence regarding this has been presented.

3. Survey method

Survey Research [6], commonly used to assess opinions and feelings, was selected as a research method for this study. The method gives an opportunity to identify characteristics of a group of individuals, and based on the results, one can compare the views of different populations.

The interview covered 11 IT companies in Tampere region, which is one of the centers of IT industry in Finland. The representatives of selected companies had participated in a certified Scrum master

Table 1
Sprint lengths in the interviewed teams.

<2 weeks	2 weeks	3 weeks	4 weeks	>4 weeks	Varying length	No Sprints
1	9	3	2	1	1	1

course. The survey is based on structured interviews of persons that play (or have played at the time) a major role in software development organizations using Scrum. Companies participating in the survey are of mixed background, some working for customer projects, and some acting in a more product-driven fashion. The range of systems includes embedded devices as well as more traditional applications, such as web-based software.

In total, 18 representatives of teams were interviewed, including project, development and quality managers as well as developers. A typical title of a person being interviewed for the survey was company development manager, who can provide as wide and covering information regarding company's software development process as possible. In addition, since the majority of the interviewed persons had senior positions, they could also provide information on how Scrum was integrated with other activities of the company.

There were 48 questions in the survey. They were composed so that the answers would cover all the objectives of this study. In the interview, additional questions were asked concerning Scrum basics whenever necessary to find out if the organization has deviated from the practice and adopted their own way of working or if the organization had not taken the practice in use in the first place. Since the results are based on personal interviews, it was possible to ensure that the persons in question were familiar with the basics of Scrum and that the interviewer understood the ways of working in the company. The complete list of questions can be found at [7].

4. Survey results

In the following, we introduce the results from the survey. In addition, we provide a brief discussion on most important observations. The results are presented in accordance to main concepts that are either used in Scrum or are common software engineering practices.

4.1. Sprints

Sprint is a time-boxed period where the actual work in Scrum takes place. The original Scrum paper [12] does not define any length for the Sprint, but among the practitioners the length of 2–4 weeks has become a common practice. Scrum guidance [15] also advises to use 2–4 week Sprints. The results of the question on Sprint length are shown in Table 1.

As Table 1 indicates, the most widely used Sprint length is 2 weeks. One company reported having a varying Sprint length meaning that if the work takes more time than planned then the Sprint is extended. Furthermore, one company reported that they did not use Sprints at all, even though they claimed that they used Scrum.

Most of the teams had experimented with the Sprint length. The most typical scenario was that the team had started with 4 week Sprints and noticed that it is too long. In most such cases 4 weeks was too long as the team could not respond to customer requests and the feedback loop became too long. In addition, two teams reported that when they had used 4 week Sprints, only two thirds of the Sprint's work was done. When they shortened the Sprint length, they could achieve a situation where all tasks were completed at the end of the Sprint. A company indicated that only 40% of too long Sprints succeed while 80% of suitable length sprints succeed.

Roughly half of the teams had tried Sprint lengths shorter than two weeks, most typically 1 week or 1.5 weeks. However, they had

Table 2
State of the testing of software after each Sprint.

Not tested	Unit tested	Intgr. tested	System tested	Acceptance tested	Delivered and installed
3	8	2	2	2	1

Table 3
Specifications documentation.

Big requirement documents	User stories	Use cases	Face-to-face	Feature descriptions
4	8	3	1	2

abandoned short Sprints as the management overhead became too large. Sprint planning and Sprint reviews was taking too much of the Sprint length. Furthermore, typically two weeks were short enough time to be able to satisfy customer's change requests.

One team had two overlapping iterations at the same time. They had iterations of 4 weeks and 2 week Sprints within the iteration. They had to take this approach as the customer did not want to participate in meetings every two weeks. Some of the teams had Sprint length of 2–4 weeks, but they had separate implementation and testing Sprints meaning that every other Sprint they implemented new features and the next Sprint was for testing those features. Of course, this approach does not produce a potentially shippable product increment each Sprint.

4.2. Testing

In Scrum, every Sprint should produce a potentially shippable product increment. In order to accomplish that, the software has to be tested before the Sprint ends. Typically this is incorporated in the definition of "done". In the interview, we asked the companies to what extent the software is tested within a Sprint. Results are shown in Table 2.

Eight teams had the software unit tested at the end of the Sprint. However, two of those teams said that they had efforts going on to make the testing more automated and the goal was to achieve automated system testing. In some cases, the system testing was carried out always in the next Sprint by a separate team. This is, however, problematic as new functionality might be already written on top of the old code which is still in the testing phase. Only three teams had the code acceptance tested within the Sprint. One of these teams was also deploying the product within the Sprint. However, they had had to limit the deployment to every other Sprint as the customer did not want to update the software so often.

According to the interviewed teams, the biggest obstacle on the road to test the code more within the Sprint was "waterfallish" thinking where testing is carried out at the end of the Sprint and the lack of automation in the testing. In addition, some of the interviewed companies were manufacturers of large machines and in these systems testing is even harder as one should test the software on the hardware before it can be shipped.

4.3. Specifications

In Scrum it is advised to document requirements as user stories or use cases in the Product Backlog. Table 3 shows the methods interviewed companies used to make specifications for their products. As the results show, four teams were still using traditional big requirement documents for specifications. Most of the teams (8) were using user stories. Two of these teams reported that they have separate UI specifications or more detailed specifications when necessary to accompany user stories. Three teams used use cases. One of these teams

Table 4
Product Owner role.

No Product Owner	Own Product Owner	Product Owner from customer	Shared Product Owner
7	7	3	1

said that they prefer use cases over user stories as use cases document context for the feature and provide some rationale for the functionality. One team had face-to-face discussions where the product was specified. After discussions with the customer and the Product Owner they documented just enough specifications that they needed to remember what was discussed. Two teams used only short feature descriptions in their Product Backlog.

As stated by Cockburn [3], the most efficient way of communicating is face-to-face. It would be advisable to have face-to-face discussions especially between the customer and the development team when making the specifications. If the requirements are communicated using written user stories, the communication is one-directional and not efficient as the development team cannot ask questions from the customer.

4.4. Product Owner

The Product Owner is responsible for maximizing the value of the product [15]. The Product Owner is also responsible for organizing the Product Backlog and creating enabling specifications for the Team. Table 4 shows the distribution of the different approaches with respect to the Product Owner.

Surprisingly, altogether seven teams reported not to have a Product Owner at all. Furthermore, three teams had a Product Owner who was customer representative, working completely off the context of the team and not understanding the dynamics of software development. This is a bit alarming as the Product Owner is responsible for optimizing Return On Investment, ROI. If the customer is in a position to optimize ROI for the company, it cannot be beneficial for the company in the long run. One team reported to have a shared Product Owner. This means that the company had one Product Owner who was managing more than one product. It was not very surprising that this team had problems as the Product Owner did not have enough time to create enabling specifications.

One team also reported to have a Product Owner team, i.e. there are multiple Product Owners that work as a team. They had evolved to this practice from the single Product Owner as the teams had been complaining that the single Product Owner did not have enough time to address the questions the teams had.

One interesting point that was discovered multiple times in the interviews was that teams who did not have a Product Owner and were working on customer projects said that having a Product Owner would not work for them. They argued that if they were building their own product, it might work better. On the other hand, multiple teams building their own product and having no Product Owner reported that the Product Owner role would work for them only if they were doing customer projects. So independently of the project type, the teams found the Product Owner role to be problematic.

4.5. Product Backlog

As already mentioned, the Product Backlog is an ordered list of things that might be needed in the product. The backlog acts as a single source of requirements for the product [15]. The Product Backlog should be ordered so that the most valuable, highest-risk or priority items are on the top (taking dependencies into account). These top items are also the most analyzed, clear and ready for development. Table 5 shows the usage of the Product Backlog in the interviewed teams.

Table 5
Use of Product Backlog.

No Product Backlog	Unordered Product Backlog	Ordered Product Backlog	Multiple Product Backlogs
3	7	7	1

Table 6
Estimating Product Backlog Items.

No estimates	Project manager or Product Owner estimates	Team estimates
2	6	10

As can be seen from Table 5, most of the teams were using the Product Backlog. Only three teams were not using it. None of these three teams reported that they have evolved away from this practice; they just had not adopted it in the first place. The largest observed problem with the Product Backlog was that in roughly half of the teams, the Product Backlog was not ordered. Considering the results of the Product Owner questions (7 teams not having a Product Owner), it is not a big surprise that the Product Backlog was not ordered in so many companies. The interviewed teams reported that the main obstacle in ordering the backlog was that the Product Owner did not have enough time for it or the Product Owner did not have required competence to order the list. However, in the interviews we also discovered that some teams were ordering the Product Backlog with the Product Owner as she was not able to do it alone. Only one team reported that they work for more than one Product Owner and select items from multiple backlogs to the Sprint.

4.6. Estimates

Work estimation is an important aspect of Scrum, enabling the efficient use of resources. In the survey we asked who is responsible for making such estimates, if anyone. We also investigated how many of the teams used hours for estimation and how many used so called estimation points. The results concerning the responsible party of the estimates are shown in Table 6.

Two of the interviewed teams did not do estimates at all. For six teams, a project manager or a Product Owner made the estimates. This number is surprisingly big and indicates that the project manager or the Product Owner dictates which items are implemented in which Sprint. Later on the question about the self-organization confirmed this conclusion. In total, ten teams produced the work estimates by themselves.

Out of the sixteen teams having estimates, eight created their estimates using hours. On the other hand, eight used estimation points or other imaginary unit to estimate the work amount. Planning poker was used by five teams out of the eight applying estimation points.

4.7. Sprint burn-down charts and velocity

Sprint burn-downs are used to illustrate remaining work in Sprint or in some cases in a release [15]. Generally the idea is to burn the chart down when a task is completed. However, different variations are often used.

In total seven teams did not use burn-downs whereas eleven teams used them. However, in interviews one team said that they have it in use, but they have never seen it. The project manager was drawing it for himself to get estimates concerning the possible failure of the Sprint. Seven teams did not use burn-downs. The main reason for not using them was that they did not see any benefits gained out of drawing the chart—it is just extra work to maintain the chart. For some teams a reason was just that there was no suitable place in the wall for the chart.

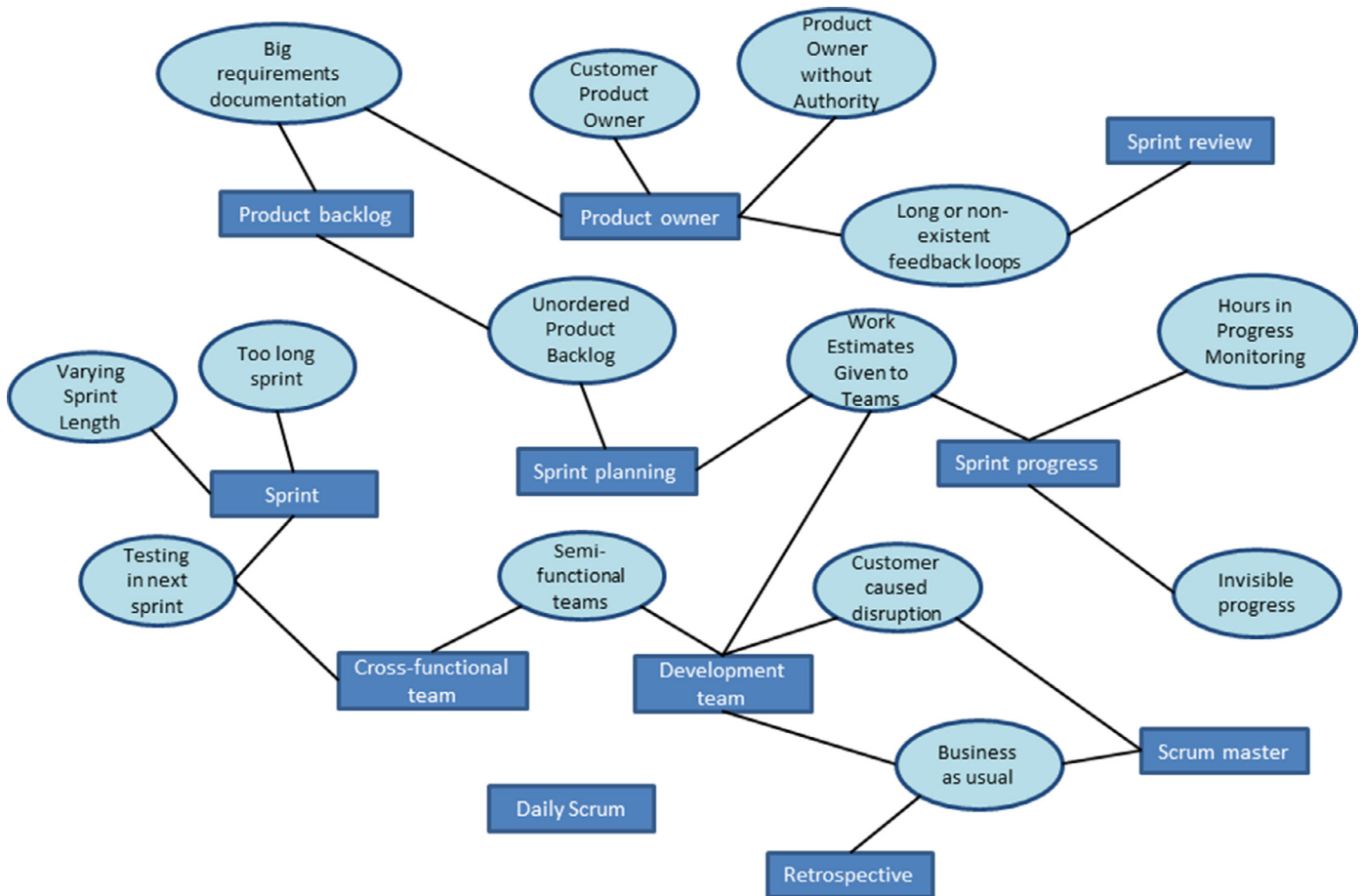


Fig. 2. Relationships of Scrum anti-patterns and Scrum concepts presented as a graph.

Out of the eleven teams who used burn-down charts, eight had partial burn-downs meaning that they used hours in the burn-down chart and once they had worked on some task, they immediately removed it from the remaining work. Only three teams burned down the graph when the task was completed.

Additionally, we asked if teams know their velocity, meaning how many work items they can do within a Sprint. Eight teams used velocity to estimate the amount of work items they can deliver in Sprints. Ten teams did not know their velocity.

4.8. Team disruption

Generally, Scrum advises to protect the Team from outside disruptions. However, in practice this might be hard to achieve as Team members have to answer phone calls, take bug reports and maintain old projects. Therefore, it is not very surprising that in thirteen cases the Teams were disrupted during the Sprint and only in five cases they were not. In one case, it was the project manager disrupting the Team, in other cases, typical cause of disruption was the customer of the current (or old) project.

4.9. Self-organization and cross-functionality of the Team

In Scrum, it is critical that Teams are cross-functional meaning that they have all the necessary expertise to deliver working software. Furthermore, the Team should be self-organized so that each Team member can choose what to work on and there is no one assigning tasks. Therefore, we asked if the companies had cross-functional Teams and if the Teams are self-organizing. In ten cases the Teams had

the necessary skills (e.g. design, coding and testing) to deliver a working product. In eight cases there were, for example, separate testing teams and therefore the Team was not cross-functional. This usually causes problems, a typical one being that the product is not ready to be delivered after the Sprint as it still needs to be tested. Furthermore, testing typically takes place during the next Sprint and, consequently, new code has often been written on top of the code being tested at the same time.

The last question about the Scrum practices concerned the self-organization of the Team, that is, to what extent the Team gets to decide what they are working on. In half of the cases, i.e. nine, the Teams were self-organized and the other half had a project manager to assign tasks to certain persons.

5. Identified Scrum anti-patterns

Based on the results of the survey and interviews, we have identified potentially harmful practices that are used in connection with Scrum, referred to as Scrum anti-patterns. A graph illustrating the relationships of Scrum anti-patterns and Scrum concepts is depicted in Fig. 2. Anti-patterns identified in the study are depicted as ellipses and Scrum concepts as boxes. Scrum concepts were identified from the Scrum guide [15] by selecting the most central elements of Scrum. In total eleven concepts were selected to be used in the graph. The relationship between a Scrum concept and an anti-pattern is illustrated with an arc indicating that the anti-pattern affects the Scrum concept. Sometimes anti-patterns affect more than one Scrum concept. We will discuss this graph in more detail in 5.3.

Table 7
Anti-pattern format.

Anti-pattern field	Contents
Name	The name of the anti-pattern
Scrum recommendation	The related Scrum recommendation
Context	Specific conditions under which the anti-pattern typically occurs
Solution	The anti-pattern (usually harmful) solution or Scrum deviation
Consequences	The consequences of using the anti-pattern
Exceptions	Possible situations in which the use of the anti-pattern could be justified
Company recommendations	Possible solutions recommended by the companies participating in the survey

5.1. Anti-pattern format

We will use the format presented in Table 7 for the presentation of the anti-patterns. The used anti-pattern format is derived from [2] with small modifications to better suit the organizational nature of the presented anti-patterns. The Scrum recommendation is taken from Scrum literature, for example [15]. The context is extracted from the results of the interview, reflecting the explanations the interviewees gave for the Scrum deviations. The consequences are partly taken from the problems noted by the interviewees and partly from Scrum literature, mostly [15].

Often the existence of another anti-pattern was observed to be one of the causes of an anti-pattern. Exceptions are situations showing up in the interviews that were considered to be “justified Scrum deviations” by the authors. In some cases, no notable exceptions could be identified. Many companies had experienced the situations described in the anti-patterns and sometimes they wanted to give advice on how to avoid the anti-patterns or how to change the course when an anti-pattern is being used. These recommendations are presented in the company recommendation section of each anti-pattern.

5.2. List of identified anti-patterns

Fourteen anti-patterns were identified based on the survey, as presented in Table 8. Anti-patterns are presented in Fig. 2. Anti-patterns are gathered around a theme: anti-patterns concerning the Product Owner role and the Product Backlog, anti-patterns concerning the Team, and finally anti-patterns related to Sprints.

The following criteria were used to accept a practice as an anti-pattern: (1) the practice is a deviation of Scrum recommendations, (2) the practice was observed at least in three interviewed teams, and (3) the practice has unfavorable consequences noted explicitly either by an interviewed team or by Scrum literature. These unfavorable consequences are described in each of the patterns in the consequences section. However, it should be emphasized that the collected empirical data do not provide any quantified evidence of the general harmfulness of the anti-patterns.

5.3. Relationships of Scrum anti-patterns and Scrum concepts

The relationship graph (Fig. 2) can be used to identify potential risk areas when adopting Scrum. For instance, from the graph it can be seen that the Team and the Product Owner are particularly sensitive to anti-patterns as both have a relationship with four anti-patterns. Sprint and Sprint progress are relatively vulnerable Scrum concepts with relationships with three anti-patterns. Consequently, the issues related to the Product Owner and to the Team are of high risk as it is easy to start using one of the anti-patterns found in this study. On the other hand, daily Scrum is not related to any of the anti-patterns which indicates that it is a low risk concept. In the interview study, almost all teams used daily Scrum practice, but none reported that it has caused problems.

Two concepts, retrospective and Sprint review, are only related to one anti-pattern. While this means that there are not many kinds of

risks related to these Scrum activities, it does not indicate that the risk is minor. For example, if a Team fails to do process improvements due to lack of sprint retrospectives, there is no continuous improvement, which is one of the key principles of Scrum. Similarly, if the feedback loop from customer to the Team via the Product Owner is long or non-existent, the Team might be building a wrong product. As the customer is not giving feedback regularly, the feedback might be delivered in the late stages of development causing massive rework and delaying the project.

To summarize, while the relationships indicate which concepts have potential risks, it does not indicate the seriousness of the risks. Using some anti-patterns might be more harmful to the efficiency of the development than others.

5.4. Scrum anti-pattern usage and company characteristics

To investigate the possible effect of the characteristics of the company on the tendency to resort to Scrum anti-patterns, we divided the companies into three categories according to their size: small (less than 30 developers), medium (30–100 developers), and large (over 100 developers). In addition, the companies were also categorized according to years of Scrum experience: less than a year, 1–2 years, and over 2 years. Table 9 shows the results.

Two significant observations can be made from the results. First, it seems that smaller companies follow Scrum principles more closely, avoiding anti-patterns. This might be due to fact that large organizations tend to have legacy practices which prevent further adoption of new practices. Lindvall et al. [8] also states that in large organizations, new practices may result in double work as old practices need to be followed when new practices are taken into use.

Second, companies having more Scrum experience tend to use anti-patterns more. On one hand this can be explained by the fact that companies might have changed their way of working as a part of continuous improvement. On the other hand, if this is the case, they have adopted anti-patterns as a part of process improvement, which, of course, is not the intention. This might need further research to find out the reasons behind the anti-pattern adoption in these companies and what kind of consequences that might have.

Two anti-patterns seem to be common despite of company size and Scrum experience: *Testing in the next Sprint* and *Customer caused disruption*. The popularity of the first anti-pattern indicates that the transformation to agile methods is particularly challenging in testing. The frequency of the second anti-pattern hints that companies do not want that the used development process affects their customer relationships and therefore let the customer disrupt the Teams.

6. Limitations

The survey was carried out in a limited region, which may influence the results. However, as different kinds of companies were included in the survey, we believe that the results can be generalized. In Survey Research, the question design is critical to get useful and valid data [6]. In this survey, the main questions, however, are directly related to Scrum basic concepts and if the interviewee understands

Table 8
Identified anti-patterns.

Name: Big requirements documentation

Scrum recommendation: The Product Backlog contains all potential items that are going to be implemented and this list is ordered by value, risk, or dependencies, for instance. Items might be described as user stories or use cases that are more easily understood as items that produce end-user value on their own right.

Context: Waterfallish thinking of the team, legacy company guidelines for requirements documentation.

Solution: Traditional big requirements documentation is produced prior to the development Sprints.

Consequences: Requirements of the system as a whole are not well understood by the team, as they are hard to find from a big documents. Requirements are not ordered by the value or other criteria; therefore it is hard for the team to know which requirements are more business critical and which not. Thus, it is harder to decide which items should be implemented next. All requirements are gathered up-front and the work might get wasted as requirements change before the features get implemented.

Exceptions: In safety-critical systems, various regulations enforce the existence of large requirements documentation and traceability to the code.

Company recommendations: Do not use an issue tracking system or project management systems as they tend to force a certain development the process. Use backlog grooming meetings every other week where the Product Owner with help from the Team creates user stories for the next Sprint but not for other future Sprints. Being agile does not mean completely giving up specifications, but large up-front documents cause a lot of work, which does not always produce value to the customer.

Name: Customer Product Owner

Scrum recommendation: The Product Owner role is necessary to ensure that the product produces value to the customer.

Context: Insufficient understanding of the role of a Product Owner, lack of persons suitable for that role, the customer does not trust the Team.

Solution: A customer representative acting as a Product Owner.

Consequences: The customer might be too busy to prepare enabling Product Backlog Items for the Team. Thus, requirements might not be clear for the Team. Organizational barrier(s) can make the communication even harder and thus make the situation even worse. Using Customer Product Owner easily leads to *Customer caused disruption* anti-pattern. Customer Product Owner often tries to dictate how the Team should implement a feature and in this way disrupt the Teams work. Customer Product Owner might try to add new items to the Sprint during the Sprint resulting in lesser commitment from the Team. In addition, when using this anti-pattern customer is optimizing the return on investment for the development company and this might be a suboptimal situation.

Exceptions: If a suitable person to represent the customer is not available, a Product Owner surrogate could be used temporarily.

Company recommendations: Try to sell the idea of having a Product Owner from your company to the customer. Alternatively, try creating a Product Owner team where the customer's Product Owner is a member.

Name: Product Owner without authority

Scrum recommendation: The Product Owner is responsible for managing the Product Backlog and deciding which items maximize the value of the product.

Context: Insufficient understanding of the role of a Product Owner, suitable persons are not motivated to use Scrum, general lack of interest towards the Product Owner role, large organizations where responsibility over products and development may be fragmented.

Solution: A development team member is promoted as the Product Owner since product managers are not interested in using Scrum.

Consequences: The Product Owner does not have the authority to decide on which items are implemented and which are discarded. The Product Owner cannot really make decisions concerning the product, and thus prioritization loses its meaning – in the end, everything must be implemented. There might be other managers above the Product Owner to make decisions concerning the product. The Product Owner is not necessarily in direct contact with the customer, and therefore the Product Owner does not know which features have value to the customer.

Exceptions: If a Product Owner with authority is not temporarily available, a surrogate Product Owner could be used for a couple of Sprints.

Company recommendations: Company adoption of Scrum is still incomplete as management does not see the value in Scrum. Try to arrange training for managers and demonstrate the benefits that would be gained from having a proper Product Owner.

Name: Long or non-existent feedback loops

Scrum recommendation: Sprint review is organized at the end of a Sprint where the Product Owner and invited key stakeholders (e.g. customer) review the product increment and provide feedback.

Context: *Product Owner without authority* anti-pattern, no direct contact with the customer. Customer is not interested in the project, just wants to see the end result.

Solution: No feedback on the made process is provided for the development Team at the end of a Sprint. The Team does not know if they are building the right product or not.

Consequences: Changes to the product at a late stage of development are likely to take place as feedback is only received when the customer starts to use the product. This anti-pattern potentially may cause a lot of rework as misunderstandings take place. The Team might start to make decisions on what to implement as they are not getting feedback or their questions answered. Using this anti-pattern may also result in building wrong features which don't have value to the customer.

Exceptions: If the Product Owner is solely responsible for the product, she can provide feedback for the Team. Otherwise other stakeholders should be involved in every Sprint review.

Company recommendations: Introduce Scrum and agile development gently to the customer. Often after the customer gets a hang of the continuous delivery and incremental development, they usually will like it and even insist that it is used in the future projects, too.

Name: Unordered Product Backlog

Scrum recommendation: The Product Backlog is ordered to give precedence for high-risk or the most valuable items.

Context: *Product Owner without authority* anti-pattern, *Customer Product Owner* anti-pattern, *Long or non-existent feedback loops* anti-pattern, insufficient competence of the Product Owner.

Solution: The Product Backlog is not ordered, but Teams select items based on their own judgment.

Consequences: As Product Backlog is unordered, the Team might be lacking vision of the risky or valuable elements of the product. As a result the Team might be building wrong features which do not have value to the customer or are just rarely used. Only features which are fun to implement get implemented as the Team starts to pick whatever they like from the backlog. Features which are hard to implement or test are left to the backlog and implemented last. This increases the risk of problems arising in the late stages of development.

Exceptions: Projects where the requirements are fixed and given up-front by the customer, and these requirements are not going to change. Some initiatives by governments and military might have such requirements.

Company recommendations: Organize Product Backlog grooming meetings every other week where the top of the Product Backlog is ordered according to the value of items and taking care of the dependencies between work items.

Name: Work estimates given to teams

Scrum recommendation: Teams should produce work estimates for themselves.

Context: Hierarchical working practices of the company. Need to know an estimate in advance on how much the product will cost.

Solution: A product manager or the Product Owner produces work estimates.

Consequences: The anti-pattern results in Teams that lack commitment: if there is slack time in Sprint, Teams do not pull features from the Product Backlog into the Sprint. On the other hand, given unrealistic Sprint goals may demotivate the Team and result in poor performance. Wrong estimates: the Team is better in estimating the effort than a single person who is not going to implement the task. Wrong estimates may lead to too optimistic or too pessimistic schedules.

Exceptions:

Company recommendations: Try out planning poker. In the beginning, it might feel uncomfortable but just keep going and it will start to make more sense. After a while, you can ask the Team to do the estimates even without poker.

(continued on next page)

Table 8 (continued)

Name: Hours in progress monitoring

Scrum recommendation: The number of items selected from the Product Backlog for a Sprint is solely up to the Team. In earlier Scrum descriptions it was advised to use story points for estimating how many user stories can be taken into a Sprint.

Context: Hierarchical organization where managers need to report how much work in hours is still left. Hours are used to calculate the price of the project and/or feature and estimates are used to find out the estimated price of a feature.

Solution: All user stories are estimated using hours.

Consequences: As a consequence of using hours in progress monitoring, estimates can become inaccurate and they are often exceeded. A lot of time is wasted estimating how much time is required for the stories and even then the estimate is often inaccurate. User stories that require studying how to implement something are hard to estimate.

Exceptions:

Company recommendations: Use story points to estimate stories. In this way, the estimates are based on relative sizing. In the first Sprint guess how many points you can deliver and after that use velocity to estimate how much can be done within a Sprint. Story points make sure that nobody expects a certain feature on a certain date.

Name: Semi-functional Teams

Scrum recommendation: Teams should be cross-functional, providing all the necessary skills to carry out the full realization of shippable products.

Context: *Testing in next Sprint* anti-pattern, lacking expertise in the company staff, division to different Teams according to the expertise.

Solution: The Team produces only certain aspects of the shippable product.

Consequences: The anti-pattern results in divided responsibility of producing a shippable product and less committed Team. Often, this also leads to staged development where the feature is first implemented by a developer and then handed over to a tester, and then back to the developer for fixing the found bugs. Hand-offs interrupt development work, or there will be wait states where no progress is made, resulting in slower feedback loops. Moreover, if testers or user experience experts are in another Team, extraneous cross-team communication is needed.

Exceptions: Highly specialized and demanding software architecture design carried out before the Sprints.

Company recommendations: Have testers and user interface designers in the same Team with developers.

Name: Customer caused disruption

Scrum recommendation: Teams should be protected against any outside disruptions.

Context: *Improper Product Owner* anti-pattern, close and interactive co-operation with a customer.

Solution: Customers are allowed to communicate directly with Team members and they can negotiate with the Team to add new features to the Sprint. Support requests from previous or parallel projects constantly interrupt the Team's work.

Consequences: Customer caused disruption interrupts the work flow of the Team, consequently reducing the efficiency of the Team. New features may crawl in to the product without the Product Owner's involvement as the customer is giving new work for the development Team directly. All work is not visible as developer is working on something customer requested, but it is not visible in the Sprint Backlog. The value stream is compromised as non-prioritized work might sneak into the Sprint.

Exceptions:

Company recommendations: Establish a question hour meaning that the customers and other stakeholders are allowed to meet and communicate with team members only during a predefined time-slot. This time slot can be one hour per day or couple of hours per week depending on the frequency of disruptions. The Scrum Master should protect the Team from disruption during all other times. In the beginning, it might be challenging, but the customer will soon learn that she should contact the Team only during this predefined time slot.

Name: Business as usual (No Sprint Retrospective)

Scrum recommendation: After the Sprint review and prior to Sprint planning, there should be a time-boxed meeting where the Team reflects on the past Sprint and creates plans for improvements that will be introduced during the next Sprint. For a four week Sprint, the retrospective should be three hours long.

Context: Insufficient understanding of agile development and Scrum, no ScrumMaster at all or the ScrumMaster role is misinterpreted as a traditional project manager.

Solution: No Sprint retrospectives organized. The Team does not reflect on Sprints to see what went well and what did not. Sometimes retrospectives may be organized but they are too short (15 minutes) and actionable items are not generated and/or they are not put in the Sprint Backlog for the next Sprint.

Consequences: As the Team is continuing business as usual, there is no continuous improvement and no changes in the productivity and efficiency of the Team. As open discussions and feedback is in the core of Scrum, failing to have retrospectives might even lead to giving up on Scrum altogether. Having *Business as usual* anti-pattern may lead to several other anti-patterns as there is no systematic way of learning what works and what doesn't.

Exceptions: If Sprints are short (2 weeks), retrospective does not need to be held after every Sprint. Retrospectives can be held every other Sprint or so.

Company recommendations: To grow agile culture of continuous improvement, organize retrospectives regularly. Hire external help to facilitate them. Try to make the feedback cycle (time between retrospectives) shorter.

Name: Invisible progress

Scrum recommendation: The progress of the work should be made visible with burn-down charts.

Context: Hierarchical working practices of the company, unsuitably distributed Team.

Solution: A product manager produces burn-down chart for herself, or no visual representation of the progress is produced.

Consequences: Invisible progress results in lacking progress awareness of the Team. The Team might think that it is not important to get all user stories finished within a Sprint. Risk of failing to deliver all features in a Sprint as the Team cannot estimate how much work they can take into the Sprint as they don't know their velocity. If it is not visible how the Team progresses, stakeholders cannot see what is happening and may not be aware of the problems the Team is having. The Scrum Master's work becomes harder as she cannot see if some user story is causing extra trouble for the Team.

Exceptions: Very small projects where people know each other, share a room, and can discuss the progress.

Company recommendations: Visualize the workflow in a suitable way. Have a burndown-chart on the wall or in the project management tool that the Teams are using. Use Information Radiator [20] to make the progress visible.

Name: Varying Sprint length

Scrum recommendation: 2–4 weeks Sprints.

Context: Lack of discipline, *Customer caused disruption* anti-pattern, *Customer Product Owner* anti-pattern.

Solution: New items are introduced to a Sprint in the middle of the Sprint. The Sprint does not end on a predefined date, but is extended so that all work can be finished.

Consequences: As there is no time boxing, visibility of the made progress gets blurrier and the possible problems in the development do not surface. No commitment to sprint goals from the Team as a Sprint just might be extended if it seems that goals will not be met. In addition, there is no exact shipping date for the product increment. This may lead to situation where ready features are laying in the version control system unused for prolonged periods. Moreover, the customer does not know when she will get the promised features. Sprint planning and Sprint review schedules need to be agreed separately causing wasted extra work.

Exceptions:

Company recommendations: Experiment with different Sprint lengths to find out which is a suitable Sprint length in the project. Stick to the length that is agreed between different parties.

Name: Too long Sprint

Scrum recommendation: 2–4 weeks Sprints.

Context: Early experimenting with Scrum, waterfall legacy of long production cycles, customers unwilling to participate in short intervals.

Solution: Using 4 weeks or even longer Sprints.

(continued on next page)

Table 8 (continued)

Consequences: In spite of longer working time for Sprints, the planned tasks tend to be unfinished at the end of a Sprint, possibly because the first weeks are not used efficiently enough and too large tasks are allocated to the Sprints. Too long sprints may prevent the customer from getting a required feature in time as priorities might change rapidly. Something more valuable can come up during the Sprint and this may lead to Customer Caused Disruption. As Sprints are long, more disruptions might occur and predictability is not so good anymore.

Exceptions: If the development must be synchronized with external work that has slower pace (e.g. hardware development), longer Sprints may be justified. Still, by-the-book Scrum could be applied internally by the development Team.

Company recommendations: Experiment with different Sprint lengths to find out which is a suitable Sprint length in the project. Start with a longer Sprint, e.g. four weeks and use that for a couple of months. Then try a shorter Sprint length. If the new length feels too short, go back to a bit longer Sprint length. One could try as short as 1.5 weeks length although that might cause too much overhead in the form of Sprint reviews and Sprint planning.

Name: Testing in next Sprint

Scrum recommendation: All testing of software is done within the Sprint that creates the software, to enable the shipping of a working product increment.

Context: Waterfallish thinking of the Team, a separate testing team, lack of testing automation, *Varying Sprint length* anti-pattern, *Too long Sprint* anti-pattern.

Solution: Testing is done in the next Sprint following a development Sprint.

Consequences: When testing takes place in the next Sprint, new code may be written on top of non-tested code. In addition, the product is not always in a “potentially shippable” state as it is still non-tested. As test results come in later, bugs need to be fixed long after the code is written. A bug might be found after several weeks after the feature was implemented. This leads to a situation where it might require some time from a developer to recall how the feature was implemented. This increases cognitive load of developers. Furthermore, hand-offs to testers and back to developers causes wait states where no value is added to the story. Hand-offs also potentially causes disruptions and additional communication is required to explain the defect to developer.

Exceptions: In an embedded system, it may be necessary to test the code together with real hardware, which might be available only at a later stage of the development.

Company recommendations: If the Team is still working in this kind of waterfall mode, freeze the code a couple of days before end of the Sprint and focus the last days only on testing. Find out how much time is required for the testing. However, work towards a mode where the stories are tested before they are considered done.

Table 9

Summary of Scrum anti-patterns in different companies.

Name	Company size			Scrum experience		
	S	M	L	<1	<2	2+
Too long Sprint		2	2	1		3
Testing in next Sprint	1	1	4	1	1	4
Big requirements document		1	3	1		3
Customer Product Owner	1	1	2		2	2
Product Owner without authority		2	3	1		4
Unordered Product Backlog	1	1	3	1	1	3
Work estimates given to teams		1	1	1		1
Invisible progress	1		3		1	3
Customer caused disruption	2	2	4	1	3	4
Semi-functional teams		1	2	1		2
Varying Sprint length		1	1		1	1
Business as usual	2	2		2	1	3
Long or non-existent feedback loops	3	2	1	1	2	3
Hours in progress monitoring	3	2	1		2	4

Scrum, the questions are likely to produce useful data. Therefore, inappropriate question design is not a major threat to the validity of this survey.

Another limiting factor is that the number of interviewed companies was relatively small, which might lead to biased sample [6]. Conducting this kind of interview study for a large number of companies would be difficult, not only because of the required amount of resources but also because companies are not necessarily willing to share information about their problems, and even use some critical resources to do it.

In the larger companies which were interviewed, the practices between teams might vary greatly, especially if teams are working in different projects. In organizations that are large and work on a single product, there seems to be more coherence in the ways of working. Still, of course, the ways of working varies between teams as teams are self-organizing at least to some extent. However, as the data set concerning large companies is small in this paper, this needs further research and can be considered as a part of future work.

Despite the above limitations, the results of this survey are in line with other survey-based studies, as will be discussed next. Still, it is likely that other anti-patterns would be probably found in other surveys, which does not invalidate the findings here.

7. Related work

As far as we know, there is no prior empirical work on identifying Scrum anti-patterns, or potentially harmful deviations from Scrum recommendations—so far, only individual observations have

been made regarding ScrumBut. In ref. [18], it is proposed that deviations from Scrum are described using (reason, workaround) pairs. Such a format can be seen as a simple anti-pattern representation.

In contrast, numerous positive properties have been associated with agile development approaches in general. Team communication—giving and receiving frequent feedback—is often reported to have increased when adopting agile methods in a company (see e.g. [10]). Also regular team meetings have been reported to support the collaboration and to give everyone better general view of the work progress. Customers are seen as valuable resources, and the customers themselves experience more active participation in the process—in contrast to the one of the weaknesses of the traditional waterfall approach where customers' current needs are not addressed until later on in the project [4]. Further benefits of agile approaches also include better process control, transparency, and improved product quality because of practices such as continuous integration and controllable-sized tasks [3]. These were confirmed in our interviews.

A recent study on agile methods by VersionOne gathered data from over 4000 survey participants. The study shows that Scrum (including also different kinds of Scrum hybrids) has 72% market share [16]. The study also lists the main benefits that companies reported being gained from the use of agile methods. The benefits include the following: (1) better control on managing changing priorities, (2) improved visibility and team morale, (3) quicker time-to-market and (4) increased productivity. The study also lists that the leading causes for failed agile projects are the following: (1) lack of experience in agile methods, (2) the organization culture does not support agile ways of working, and (3) external pressure to use traditional waterfall practices. All these problem areas were also identified in our interviews. In addition, the report lists barriers to further adoption of agile practices. The most significant barriers are: (1) lacking ability to change organization culture, (2) general resistance to change, (3) availability of skilled personnel, and (4) lack of management support. These all are also in line with our results from the interviews, giving further confidence towards the validity of our results. However, since our results have been gathered with personal interviews, we believe the results have more depth than what can be collected with questionnaires only. In particular, the potential problems and deviations are easier to identify in personal interviews.

Another survey conducted by Salo and Abrahamsson [11] gathered data from 35 projects from 13 software organizations from eight different European countries. The survey was carried out using web-based questionnaire, and the goal was to find out the level of adoption and the usefulness of agile methods such as Scrum and XP. The study

was carried out in 2006, so the data is rather old. This can be seen in the results: Scrum was used only in 27% of companies who participated in the study. When compared to the survey by VersionOne [16], the market share differs radically. Furthermore, Salo and Abrahamsson [11] report that the most popularly used Scrum practice was the Product Backlog. However, our results imply that the situation has changed and the most widely used Scrum practice is constituted by Sprints, arguably reflecting the fashion the actual work is organized. Moreover, also other practices that are intimately associated with actual development activities, such as the daily Scrum, were widely used in the companies participating in the study.

A summarizing comparison of different benefits and issues of agile methods has been given by Petersen [9]. In addition to Petersen's work, the benefits and challenges of agile methods have also been studied by Dybå and Dingsøyr [5]. Their systematic article reviews several virtues and benefits of these methods as well as the associated problems and challenges. In general, these results fall along the lines of our survey as well.

8. Future work

In this paper we only addressed basic Scrum practices on the development side, the results are inconclusive when considering the integration of Scrum practices in the activities of companies. Consequently, there are several directions of future work.

Firstly, to gain better confidence on the results, the survey should be conducted on a wider range of companies. Preferably, companies from several geographical areas are to be involved in this study, representing different software development domains and cultures, to provide a more comprehensive data set. We trust that this also gives us an improved understanding regarding the relations of anti-patterns.

Secondly, a deeper analysis of problematic areas in Scrum—in particular the role of the Product Owner—is needed to better understand the root causes of some of the issues associated with anti-patterns identified in this paper. Deeper analysis could also provide insights on the seriousness of various anti-patterns and help to categorize them further. This work has already started with discussions with some company representatives who have participated in our survey.

Finally, issues such as synchronizing the activities of individual Scrum teams into a larger system e.g. in the form of Scrums [13] were not considered. Such research would be best executed in cooperation with other research institutes to gain access to a large enough number of case companies from various domains, countries, and continents.

9. Conclusions

Due to the rapid pace of new discoveries in the field of software engineering, new practices are commonly taken into use before conclusive evidence on their results is available. It is only after a while when the new practices mature and get adapted to the actual needs of the different companies, and the problematic elements of the practices can be identified. At this point, it is possible to gather empirical data on the use of the practices.

In this paper, we have introduced the results from a series of interviews in software industry, where the goal was to understand how Scrum is deployed in practice, and especially what are the most notable deviations from Scrum recommendations, as well as the rationale behind these deviations. A central goal of this work was to express the deviations in a systematic form using the anti-pattern

concept. The list of anti-patterns derived in this work gives a good picture of Scrum practices that are in general found difficult to follow by companies, for various reasons. This work does not, however, provide empirical evidence of the general harmfulness of using these anti-patterns. Indeed, in some cases coming up in the interviews, using certain anti-patterns seems to be well justified. Finding such evidence calls for further research based on data gathered from a more extensive set of companies. The results of our work can be exploited by practitioners taking Scrum into use in a company, to pay attention to seemingly attractive Scrum deviations that may nevertheless have unpleasant consequences. The list can also be used by researchers to understand the problematic aspects of Scrum in real life, and to provide guidance and practice refinements to avoid resorting to these anti-patterns.

Acknowledgements

This work is partly funded by the Finnish Funding Agency for Technology and Innovation in the context of project Sulava. The authors wish to thank all the interviewed companies for their cooperation.

References

- [1] Agile alliance: manifesto for agile software, Available at <http://agilemanifesto.org> 2001 (accessed 1.03.15).
- [2] W.J. Brown, et al., *Antipatterns - Refactoring Software, Architectures, and Projects in Crisis*, Wiley, 1998.
- [3] A. Cockburn, *Agile Software Development*, 1st ed., Addison-Wesley Professional, 2001, p. 256. ISBN 0-201-69969-9.
- [4] J.O. Coplien, G. Bjørnvig, *Lean Software Architecture for Agile Software Development*, Wiley, 2010, p. 376. ISBN 978-0-470-68420-7.
- [5] Dybå T., T. Dingsøyr, Empirical studies of agile software development: a systematic review, *Inf. Softw. Technol.* 50 (9–10) (2008) 833–859.
- [6] S. Easterbrook, J. Signer, M.-A. Storey, D. Damian, Selecting empirical methods for software engineering research, in: F. Shull, J. Singer, D. Sjöberg (Eds.), *Guide to Advanced Empirical Software Engineering*, 2008. ISBN 978-1-84800-043-8.
- [7] Eloranta V.-P.: Interview questions. Available at http://www.elisanet.fi/weellu/scrum/interview_questions.html (2015) (accessed 28.10.15).
- [8] M. Lindvall, D. Muthig, A. Dagnino, C. Wallin, M. Stupperich, D. Kiefer, J. May, T. Kahkonen, Agile software development in large organizations, *Computer* 37 (12) (Dec. 2004) 26, 34.
- [9] K. Petersen, C. Wohlin, A comparison of issues and advantages in agile and incremental development between state of the art and an industrial case, *J. Syst. Softw.* 82 (9) (2009) 1479–1490.
- [10] M. Pikkarainen, J. Haikara, O. Salo, P. Abrahamsson, J. Still, The impact of agile practices on communication in software development, *J. Empir. Softw. Eng.* 13 (3) (2008) 303–337.
- [11] O. Salo, P. Abrahamsson, Agile methods in European embedded software development organisations: a survey on the actual use and usefulness of extreme programming and Scrum, *IET Softw.* 2 (1) (2008) 58–64.
- [12] K. Schwaber, SCRUM development process, in: *Proceedings of 10th Annual Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 1995, pp. 117–134.
- [13] J. Sutherland, Agile can scale: inventing and reinventing Scrum in five companies, *Cutter IT J.* 14 (12) (2001).
- [14] Sutherland J.: ScrumBut Test aka The Nokia Test. Available at <http://jeffsutherland.com/scrumbutttest.pdf> (2010) (accessed 15.06.15).
- [15] Sutherland J., Schwaber K.: The Scrum Guide—The Definitive Guide to Scrum: The Rules of the Game. Available at <http://www.scrum.org/storage/scrumguides/Scrum%20Guide%20-%202011.pdf> (2011) (accessed 15.06.15).
- [16] VersionOne 7th Annual State of Agile Survey. Available at <http://www.versionone.com/pdf/7th-Annual-State-of-Agile-Development-Survey.pdf> (2012)
- [17] V.-P. Eloranta, K. Koskimies, T. Mikkonen, J. Vuorinen, Scrum anti-patterns—an empirical study, in: *Proceedings of 20th Asia-Pacific Software Engineering Conference (APSEC)*, 2013, pp. 503–510.
- [18] Scrum homepage. ScrumButs and Modifying Scrum. Available at <https://www.scrum.org/scrumbut> (accessed 10.03.15).
- [19] Jeffries R.: Fractional Scrum, or “Scrum-But”. *AgileAtlas*. Available at <http://agileatlas.org/articles/item/fractional-scrum-or-scrum-but> (accessed 16.12.13).
- [20] A. Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams* (2004).